

Chapter 4

Our work

The main objective of our work was to develop an interactive segmentation method, which utilised weakly annotated data. We decided to adopt a similar approach to some of the works analysed in Section 3.3, which were based on an auxiliary correction network that refined the rough segmentations predicted from a segmentation network.

In general, our work can be split into two main parts. In the first part, we implemented a simple segmentation model, which was trained in a fully-supervised manner on a small portion of the dataset. In the second part, we trained a correction model to correct local errors from small cuts. These cuts are extracted from the initial segmentation based on user-defined clicks. To make the training of the correction network feasible, we also developed a method to simulate user clicks and to generate erroneous segmentations. Lastly, we employed the pre-trained correction network to refine the wrongly segmented regions from the segmentation network. A diagram describing our method can be seen in Figure (4.1).

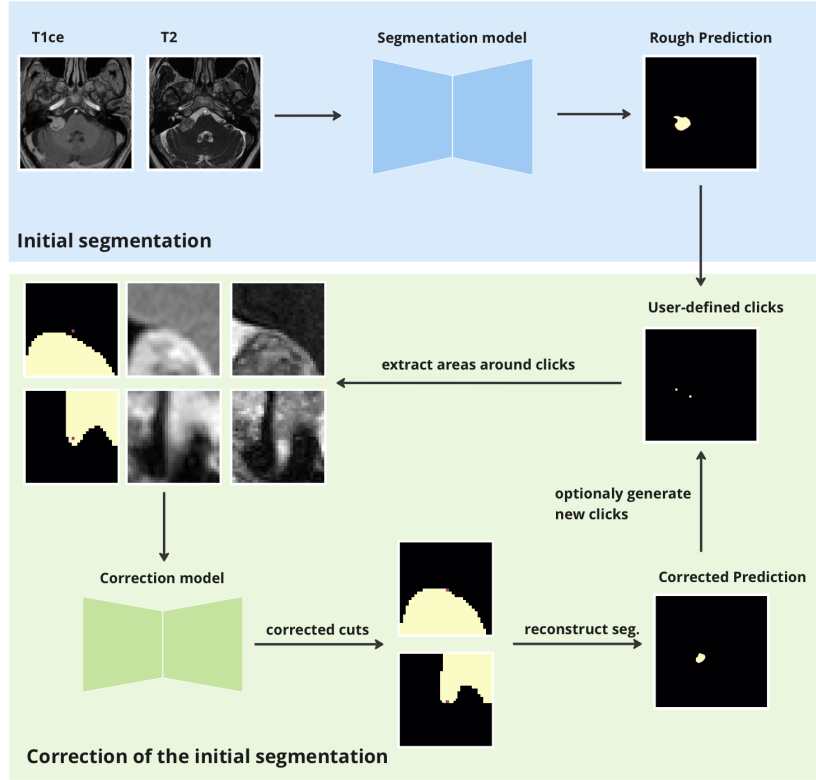


Figure 4.1: Diagram of our proposed method consisting of two main parts. First part generates initial segmentation and the second part employs an auxiliary correction network which refines the segmentation based on user-defined clicks.

4.1 Dataset

In our work we utilise the publicly available Vestibular Schwannoma (VS) dataset¹, which consists of 242 brain MRI scans in T1ce and T2 sequences [3]. Additionally to the scans, the dataset also includes annotations of the VS for each scan. An example of one slice in each sequence and the corresponding annotations can be seen in Figure (4.2). Originally the scans come in the DICOM format and the two sequences are not co-registered, where the T1ce sequence usually has more slices than the T2. The curators of the dataset provide pre-processing scripts² to

¹<https://www.cancerimagingarchive.net/collection/vestibular-schwannoma-seg/>

²https://github.com/KCL-BMEIS/VS_Seg/tree/master/preprocessing

convert the data into the NIfTI format and co-register the sequences.

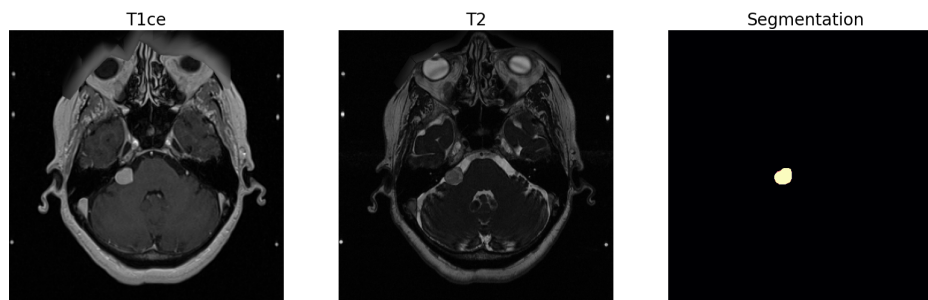


Figure 4.2: Sample T1ce, T2 sequences and the corresponding Vestibular Schwannoma segmentation from the dataset.

While analysing the dataset, we came across a couple of issues that needed to be addressed. First of all, we found out that the final volumes come in different shapes. In particular, there are 191 scans with shape 448x448x80, 48 scans with shape 384x384x40, one with 384x384x80, one with 448x448x70, and one with 384x384x20. Another issue we came across is that the data are highly unbalanced, where the tumour voxels only take up 0.006 of the total space.

4.2 Data preprocessing

As part of data pre-processing, we cropped and resized the sequences from their original shapes to 40x256x256 to deal with the shape variety and also to make training feasible due to high memory requirements. The cropping removed most of the background from the scans and we also decided to remove some slices from the sequences since the average depth of the tumour was just around 10 slices with the maximal depth being 20. The depth cropping was done in such a way that the tumour was somewhere in the middle of the new volume. However, one scan has a depth smaller (20) than the selected depth (40), where in this case we padded the first and last 10 slices with zeros.

The adjusted sequences were further normalised using a Min-Max scaler and stacked together, so the final tensors had a shape of $(b, 2, 40, 256, 256)$ for the input images and $(b, 2, 40, 256, 256)$ for segmentation masks, where b represents the batch size. An example of the preprocessed sequences and the corresponding segmentation can be seen in Figure (4.3).



Figure 4.3: An example of the preprocessed sequences and segmentation.

Additionally, to verify that preprocessing did not intervene with the tumour, we summed all preprocessed segmentations. The visualisation can be seen in the Figure (4.4), where we verified that the tumours remained intact.

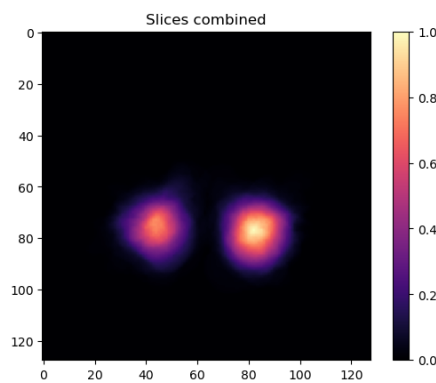


Figure 4.4: Combined segmentations.

4.3 Proposed method

As we have already mentioned, our method consists of two main parts, the generation of an initial segmentation and the subsequent correction of this segmentation in the second part, as can be seen in the diagram in Figure (4.1).

The first part is composed of a pre-trained segmentation model that generates initial rough segmentations. In this part, we experimented with different loss functions, in order to find a loss function that would allow us to use as little fully-annotated images as possible, while producing segmentations that are still satisfactory.

In the second part, the initial segmentation is cut on the basis of user-defined clicks into small cutouts. These are then sent to the pre-trained auxiliary correction model that refines them. For the training of the correction network we used erroneous segmentation cuts that we generated from the ground-truth labels. In this part, we experimented with different network architectures, loss functions, and different modifications of the generated training cuts.

Finally, the extracted cuts are reconstructed back into the initial volume, where the user can optionally generate new clicks to further improve the segmentation.

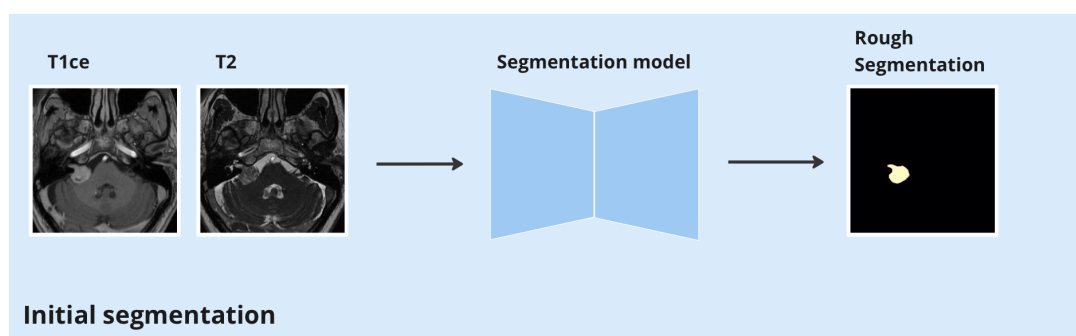


Figure 4.5: Generation of the initial segmentation.

4.4 Initial segmentation model

In this part we designed a method for segmentation of the vestibular schwannoma from both available MRI sequences. The model was trained in a fully-supervised manner, where the purpose of the segmentation model was to generate initial segmentations. The general process for this part is shown in Figure (4.5).

4.4.1 Architecture

We chose to use the 3D U-Net architecture [9], as it has proven to be very successful in this domain [17, 18, 19]. A diagram showing our architecture can be seen in the Figure (4.6).

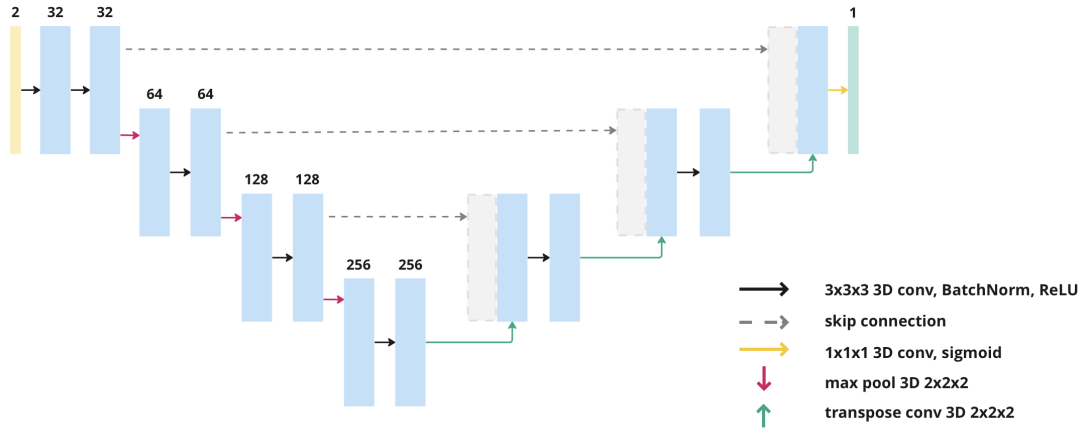


Figure 4.6: U-Net architecture of the segmentation model.

The layout of our architecture follows the standard U-Net, with three stages in both the encoder and the decoder. Each stage consists of two convolutional layers with 3x3x3 3D convolutions and zero padding followed by BatchNorm and the ReLU activation function.

The downsampling operation is performed by 3D max-pooling layers with pooling size 2x2x2 and stride 2. On the contrary, the upsampling of the feature maps is

achieved by transposed 3D convolutions with kernel size $2 \times 2 \times 2$, stride 2 and zero padding.

The input of the network has 2 channels and consists of stacked T1ce and T2 sequences. Subsequently, the channels of the feature maps double through the downsampling path, starting from 32 and ending with 256 in the bottleneck. In the upsampling path, the number of feature channels is halved. Since we are performing binary segmentation, the last layer is a $1 \times 1 \times 1$ convolution with one channel followed by the sigmoid activation function, which outputs the segmentation.

4.4.2 Training of the segmentation model

For model training, we randomly split the data into three sets, where 170 samples were in the training set and 36 samples were in the validation set. The remaining 36 images were left for testing of our models. We used both T1ce and T2 sequences for training of the network. These sequences were stacked together and used as input to the segmentation network.

We trained the baseline model for 50 epochs with batch size 2 and Adam optimiser with an initial learning rate set to 0.001, which decreased on plateau by a factor of 0.5.

As for the loss function, we decided to base our experiments in this part of the work around finding the most suitable loss function for this task. We in particular paid attention to loss functions that are suitable for segmentation from unbalanced datasets.

For monitoring the training process, as well as for the final evaluation of the model, we decided to use the dice score coefficient (DSC), which measures the overlap between the computed segmentation and the ground truth. The equation

describing the DSC can be found in the analytical part of the document (eq. 2.3).

4.4.3 Results

Since the VS dataset is fairly unbalanced, we decided to first experiment with multiple loss functions to find the one most suitable for our task. In this experiment, we used the whole training dataset. The results from both training and testing can be seen in Table (4.1).

Overall we experimented with weighted binary cross-entropy (bce), dice loss, the combination of dice loss and bce, focal loss, tversky loss and the combination of focal and tversky loss.

	Training	Validation	Testing
Weighted binary cross-entropy	0.7546	0.7232	0.7091
Dice loss	0.8720	0.8408	0.8329
Combination of dice loss and bce	0.8901	0.8378	0.8432
Tversky loss	0.8667	0.8248	0.8150
FocalTversky loss	0.9243	0.8838	0.8702

Table 4.1: Training, validation and testing DSC for models trained with different loss functions.

The **FocalTversky loss** performed the best in this experiment, where the model trained with this loss function achieved a test DSC of **0.8702**. Surprisingly, the Dice loss function also performed relatively well and achieved the test DSC of 0.8329. An example segmentation by the model trained with FocalTversky loss can be seen in Figure (4.7).

In our case, however, we rather wanted the model to use fewer fully-annotated images, so in another experiment we employed the two best performing loss functions on smaller training sizes. The purpose of this experiment was to find the

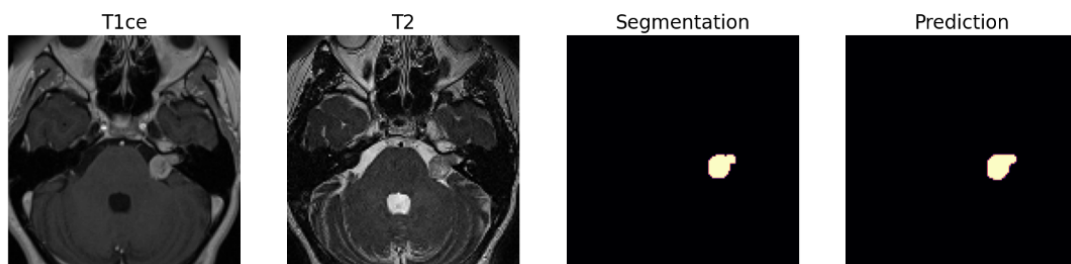


Figure 4.7: An example segmentation from the segmentation model. On this particular scan the model achieved DSC of 0.9034.

least amount of data that we can use for the model to produce segmentations that are still satisfactory (above 0.75 DSC).

	Training Size	Training	Validation	Testing
Dice loss	32	0.9071	0.6553	0.6313
FocalTversky loss	32	0.8895	0.7125	0.7032
Dice loss	64	0.9209	0.7765	0.7783
FocalTversky loss	64	0.9204	0.8017	0.8030
Dice loss	128	0.9291	0.8607	0.8643
FocalTversky loss	128	0.9333	0.8736	0.8612

Table 4.2: Training, validation and testing DSC for models trained with Dice and FocalTversky loss functions on different training sizes.

As can be seen in Table (4.2), the best model was trained using the FocalTversky loss on a training set of 128 images. We decided that 128 images represented the maximum number of images we wanted to use, as this already exceeded 75% of our training set. Graphs showing the training of these models are provided in the Appendix ??.

Models trained on a smaller amount of training data were visibly prone to overfitting, as indicated by the high differences between training and validation DSC. This did not pose a problem for us, as the use case of our method is to use an additional network to correct inaccurate segmentations. With an overfitted model, we can more accurately test the correction network capabilities. As a result, we

decided to use the model trained on 64 images and with the FocalTversky loss function as **the segmentation model**.

4.5 Correction model

In the second part of our work, we implemented a correction network, which was designed to correct the segmentation errors of the given segmentation mask. The process of correcting the initial segmentation can be seen in Figure (4.8).

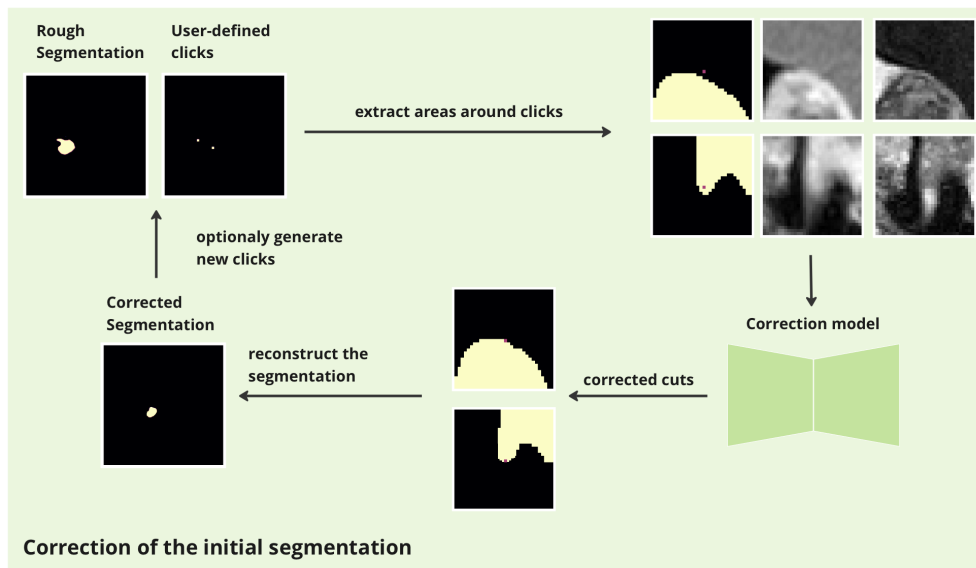


Figure 4.8: Diagram depicting the correction of the initial segmentation.

Our strategy for this part involved extracting small areas from the segmentation and training the network to correct the defects in these small cutouts. This approach allowed the network to focus solely on the local area of the faulty segmentation, which proved beneficial for the networks overall performance.

These cutouts are generated after the user clicks on an area of the segmentation that they want to correct. After that, a small area around the click is extracted and sent to the correction network.

4.5.1 Training of the correction model

For the training of the network to be feasible, we first needed to design a method to simulate user clicks and to automatically generate faulty segmentations. The process of training the correction network is depicted in Figure (4.9). The process consists of utilising the preprocessed data to generate a set of training cuts and true cuts. The training cuts are used as input to the network, while the true cuts are used to evaluate corrected cuts returned by the network.

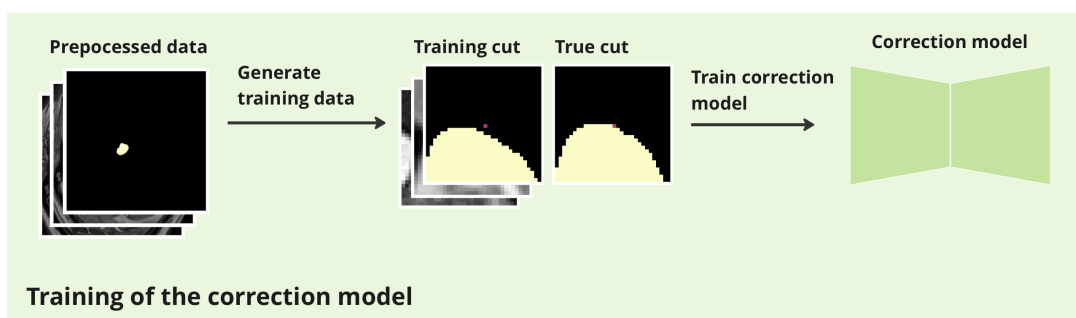


Figure 4.9: Diagram depicting the training of the correction network.

4.5.2 Generating training data

The generation of data for the training of the correction network consists of four main steps, which can be seen in the diagram in Figure (4.10). In the first place, the input volumes are preprocessed, where we make use of the preprocessing pipeline as described in Section 4.2.



Figure 4.10: Diagram of the pipeline for generating training data.

The preprocessed volumes are used to generate border clicks (the process of gen-

erating border clicks is described in 4.5.2.1). These clicks are then used to cut an area from both the MRI sequences and the segmentation mask. The cut is made around the click, with the click being in the middle of the acquired cutout. We tried different sizes of the cutouts, from which cuts with size 48 (width and height) performed the best (Table 4.4). Afterwards, we create copies of the cuts and modify them to resemble faulty segmentations (the whole process is described in 4.5.2.2). These unmodified and erroneous cuts form the dataset used for training of the correction network.

4.5.2.1 Generating clicks

The clicks are generated from the provided segmentation mask by first using morphological erosion to get a border around the segmented object (the original segmentation - erosion). Subsequently, a set number of points are randomly selected from the acquired border. To ensure that the points are not generated too close to each other, they are selected based on the distance between them. This distance is calculated using the Euclidean distance.

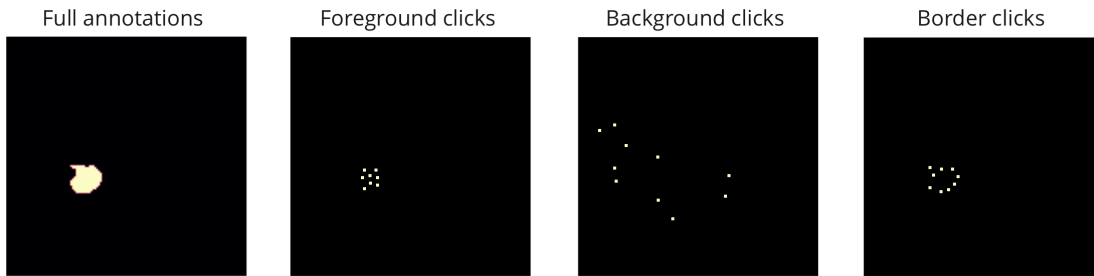


Figure 4.11: An example of generated background, foreground and border clicks.

Similarly, we can also generate other types of clicks, such as background (dilatation - the original segmentation) and foreground (selected from the original segmentation). An example of generated clicks can be seen in Figure (4.11).

4.5.2.2 Simulating errors

For simulating segmentation errors, we applied morphological dilatation and erosion on acquired segmentation cutouts. To mimic oversegmentation, we used dilatation; on the other hand, to simulate insufficient segmentation, we used erosion. To prevent the destruction of the segmentation, we reduced the number of iterations for erosion when applied on smaller segmentations. An example of generated erroneous segmentations can be seen in Figure (4.12).



Figure 4.12: An example of generated cuts (top row) and cuts with generated errors (bottom row).

To ensure sparsity between the generated errors, the morphological operations were selected randomly. As a form of regularisation we also left some of the cuts unchanged, where with 20% probability, the error generation would be skipped for the given cut.

4.5.3 Simulating user interaction

In order to make the testing of the correction network faster, we developed a method to automatically generate clicks in the areas of the segmentation that need to be refined. The clicks are generally added in the area where there is the biggest

difference between the original (ground truth) segmentation and the predicted segmentation. The process can be described by the following pseudocode:

Algorithm 1: Simulating user clicks

Data: y_true, y_pred

Result: $clicks$

$border \leftarrow y_true - erosion(y_true);$

if $y_true[y_true == 1] > y_pred[y_pred == 1]$ **then**

$dst \leftarrow euclidean_distance_transform(1 - y_pred);$

else

$dst \leftarrow euclidean_distance_transform(y_pred);$

end

$weighted_border \leftarrow dst \times border;$

$click_coordinates \leftarrow where(weighted_border == weighted_border.max());$

$clicks \leftarrow select_clicks(click_coordinates);$

4.5.4 Loss function

For this task, we also designed a simple loss function. The loss consisted of a dice coefficient that is scaled by a weight map (w). The weight map ensures that the middle pixels have higher weights, so the network would therefore be more penalised for segmenting them incorrectly. Since the cuts are made around the middle pixel, weighting the dice coefficient in this way should improve the accuracy of the final segmentation compared to normal dice loss. This "correction" loss (L_{corr}) can be defined as:

$$L_{corr} = 1 - 2 \frac{\sum_{i=1}^W \sum_{j=1}^H (p_{i,j} \cdot y_{i,j} \cdot w_{i,j})}{\sum_{i=1}^W \sum_{j=1}^H ((p_{i,j} + y_{i,j}) \cdot w_{i,j})} \quad (4.1)$$

where the numerator computes the intersection between the two volumes, and the denominator computes the union. Both intersection and union are performed by multiplying or summing the volumes along the spatial dimensions.

4.5.5 Architecture

For the correction network, we chose to use a 2D Attention U-Net with attention blocks in the decoder. The use of the attention mechanism proved to be beneficial for the performance of the network (as can be seen in Table 4.3). The architecture of the used network is shown in Figure (4.13).

Apart from the additional attention blocks, the network follows the standard U-Net architecture. In this case, we used 3-channel input, which consisted of stacked cuts of the T1ce and T2 sequences and the incorrect segmentation. The output of the network has 1 channel and represents the corrected segmentation cut.

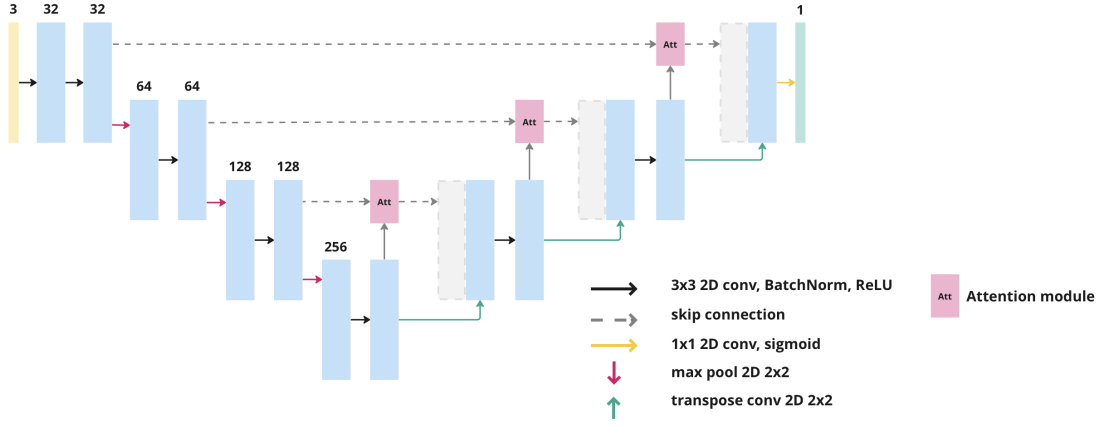


Figure 4.13: Attention U-Net architecture used for the correction model.

For the attention mechanism, we used spatial attention as described by Oktay et al. [14] and its architecture can be seen in Figure (4.14).

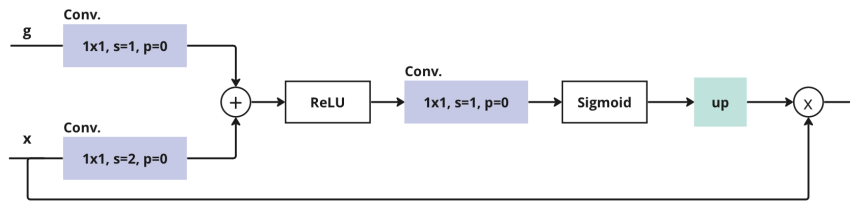


Figure 4.14: Architecture of the attention block used in the correction model.

The first two convolutions in the attention module adjust the number of channels of both inputs. The $\times$ convolution also downsamples the input since the skip connection has bigger dimensions than the gating signal. These adjusted feature maps are then added together and run through the ReLU activation function. Subsequently, we apply another convolution, which reduces the number of channels to 1 and scales the output values to a range between 1 and 0 using the Sigmoid function. The resulting coefficients are upsampled to match the dimensions of the skip connection and then multiplied by the input skip connection.

To help with regularisation, we also added dropout layers with 40 % change after each ReLU in the convolutional blocks.

4.5.6 Training setup

For the training of the correction network we kept the same data split as for the training of the segmentation network (i.e., 170 for training, 36 for validation and 36 for testing). In this part, we used the incorrect segmentation and both T1ce and T2 sequences as input to the network. For generating the dataset, we used all of the training and validation scans. Since the scans are volumetric and there can be multiple clicks on each slice, the final number of training images was; 8128 for training and 2212 for validation.

All experiments were trained for 50 epochs with a batch size of 4. We used the same optimiser settings as in the segmentation network training (4.4.2).

In this part of the work we performed first correction trainings and experimented with data generator settings, where we tried different cut sizes. Furthermore, we also tried to verify the benefits of the attention blocks. In addition, we performed experiments on both dice and correction loss functions.

4.5.7 Testing of the correction network

The testing of the trained correction models was done on cuts created from scans in the testing set. At first, we generated segmentation for the selected scan. For this, we utilised the trained segmentation model. First, we generate clicks for the acquired prediction (explained in 4.5.3). Then we used the clicks to cut the generated segmentation and MRI sequences. These acquired cuts represent the test dataset.

To ensure that all models will have the same conditions, we set a custom seed for the click selections, so it will always return the same set of clicks.

4.5.8 Results

At first we wanted to confirm that the use of spatial attention and redefined dice loss have beneficial effects on the training of the model. For this we conducted 4 trainings with alternating the dice and correction losses as well as architecture with or without the spatial attention. The results of these experiments are shown in Table 4.3. For these experiments, we used the same cut size, which was set to 48.

	Testing
dice loss, attention blocks	0.8328
correction loss, attention blocks	0.8417
dice loss	0.8072
correction loss	0.8251

Table 4.3: Testing DSC for models with / without the attention blocks and trained on either dice or attention losses.

We were able to confirm that the inclusion of the attention blocks had a positive effect on the performance of the model, where the testing DSC slightly improved in both cases. Similarly, the correction loss also performed better than the basic

dice loss function. The best model was therefore trained with these settings and achieved testing DSC of 0.8417.

In the following experiments, we tried to use different cut sizes to find the most suitable one. We experimented with four different cut sizes; 16, 32, 40 and 48. We did not include cutouts with sizes over 48 as then the cutout would not capture the local area of the tumour, as the cutout would usually include the whole tumour. Testing DSC for different cut sizes can be seen in Table 4.4.

	Testing
correction loss, 16-sized cuts	0.7042
correction loss, 32-sized cuts	0.8042
correction loss, 40-sized cuts	0.8350
correction loss, 48-sized cuts	0.8417

Table 4.4: Testing DSC for models trained on cutouts of different sizes.

Based on the results, the model trained on 48-sized cuts performed the best. Overall, the performance was improving with larger cut sizes. The model trained on 16-sized cuts performed significantly worse, which can be attributed to the fact that the sequence cutouts were, in this case, too small and the details were too distorted.

With these experiments, we were able to find the base configuration for the correction model. Subsequently, all of the following models were trained with the correction loss function, attention modules, and 48-sized cuts. An example of a corrected segmentation of the best trained model in this stage can be seen in Figure (4.15). On the particular scan, the model achieved a DSC of 0.7228. Compared to the segmentation predicted by the segmentation model (0.5727 DSC) the correction improved the segmentation by 0.1501.

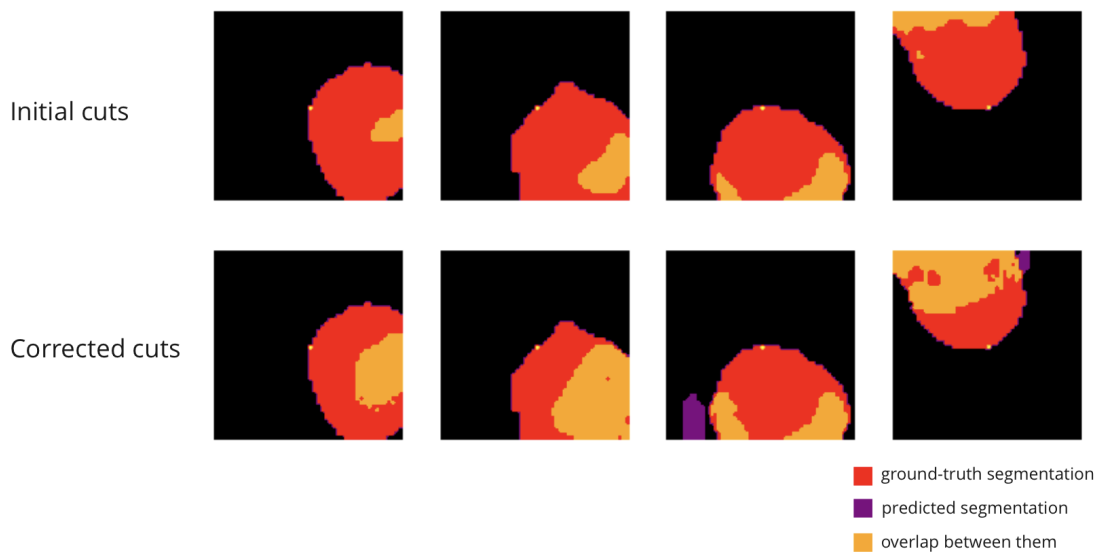


Figure 4.15: An example of corrected cuts produced by the attention U-Net (bottom row) compared to the original segmentation (top row). In this particular case, the corrected cuts achieved DSC of 0.7228.

4.6 Multi-encoder U-Net

From the results of the previous experiments, we hypothesised that the network did not adequately utilise the information from the MRI sequences. In [59, 60], the authors utilised multi-encoder segmentation architectures to better incorporate different modalities into the training of their network.

Inspired by their work, we decided to try this modification as well, since the additional encoder might help the network to better extract the information from these modalities. In our case, we decided to use two separated encoders. One for the incorrect cut and the other for the stacked MRI sequences.

Apart from the modification of the network architecture, we used the same training process as in the previous experiments. For the training of the network, we utilised the correction loss and 48-sized cuts, as this combination performed the best in previous experiments (Table 4.4).

4.6.1 Architecture

The multi-encoder U-Net architecture mainly differs from the standard U-Net in the downsampling path. In this case, the downsampling path of the network includes a supplementary encoder for the additional modalities. In our case, we decided to use one encoder for the incorrect segmentation cut and one additional encoder for the stacked MRI sequences. The last downsampled feature maps from these two encoders are concatenated before the bottleneck.

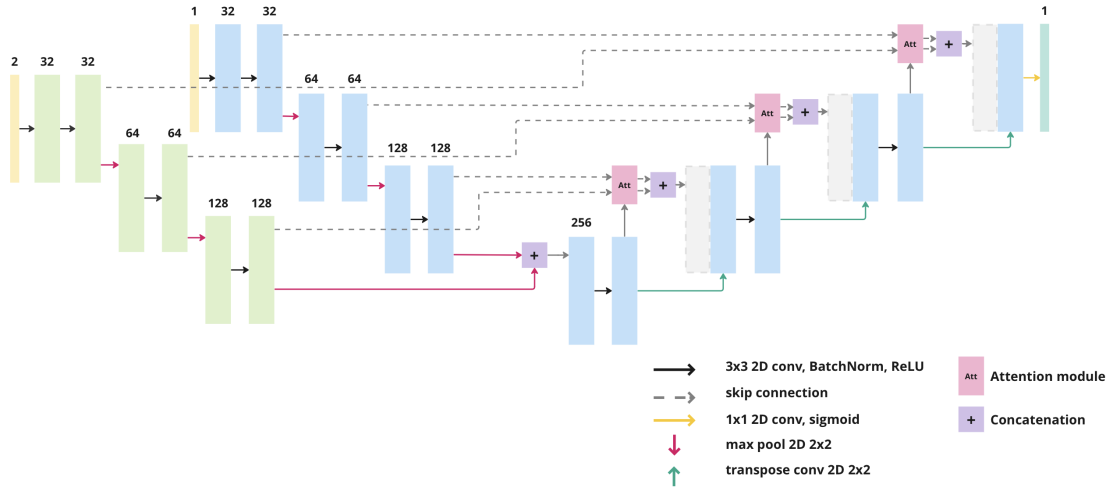


Figure 4.16: Modified correction network architecture with two encoders.

In the upsampling path the skip connections are first inputted into separate attention modules (in the figure shown as attention module with two inputs). Opposed to [60], we only compute spatial attention for each skip connection and concatenate the resulting feature maps. The concatenated feature maps then continue to the corresponding upsampling block, where we first adjust the number of channels before concatenating them with the upsampled feature maps. In this case, we use the same spatial attention module as described 4.6.1.

4.6.2 Results

In this experiment, we wanted to compare whether the use of an additional encoder helped the model to better utilise the information provided from the MRI sequences. The results of this experiment can be seen in Table 4.5. In this case, we compared the trained model with the best model we have trained so far (trained with the correction loss on 48-sized cuts).

	Testing
initial cuts	0.7630
multi-encoder U-Net	0.8687

Table 4.5: Testing DSC for the best attention U-Net based model and for the model trained with the multi-encoder U-Net architecture.

As can be seen from the results, the additional encoder slightly improved the models performance and achieved testing DSC of 0.8687, compared to the 0.8417 achieved by the previous best model. A similar improvement can also be seen in the example of corrected cuts in Figure (4.17). Where the correction slightly improved in comparison to the attention U-Net based model (from 0.7228 to 0.7884).

4.7 Using volumetric cuts

Furthermore, to improve the networks performance, we attempted to also utilise volumetric cuts, as we believed that the additional information could be beneficial to the accuracy of the network.

To generate the training cuts, we followed the same pipeline as described in Section 4.5.2. The only difference was that, except for the width and height of the volume, we also cut the depth. With this approach, only one slice of the resulting cutout correctly matched with the provided click. To account for the additional slices, we made some modifications to the correction loss. The errors were generated for

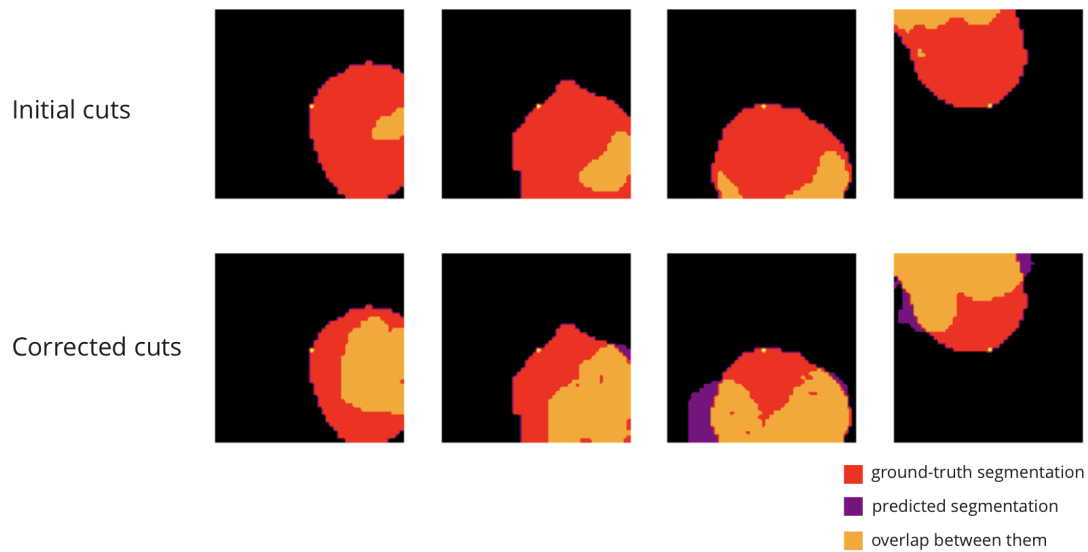


Figure 4.17: An example of corrected cuts produced by the multi-encoder (bottom row) compared to the original segmentation (top row). In this particular case, the corrected cuts achieved DSC of 0.7884.

all of the slices in the given cut. An example of generated volumetric cuts can be seen in Figure (4.18).

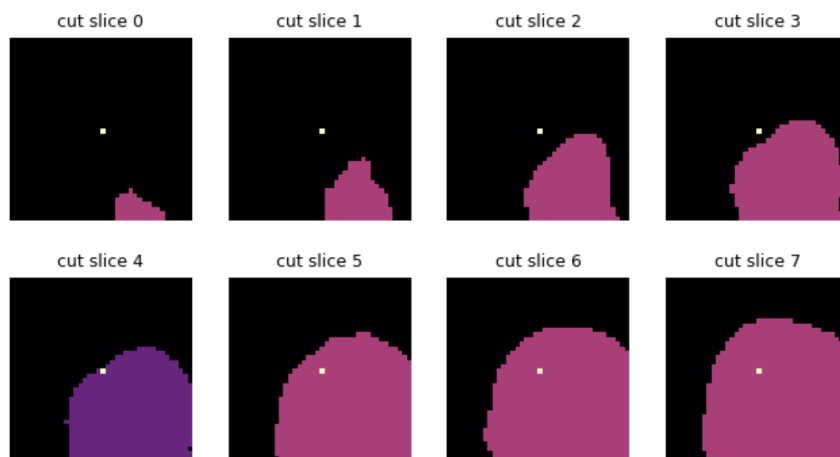


Figure 4.18: Example of generated volumetric cuts with depth=8, where the middle slice is denoted by darker colour.

For training, we used the multi-encoder architecture as described in 4.6.1 and only

changed the dimensions of the individual operations (convolutions, downsampling and upsampling). As for the experiments, we tried to train the network with different cut depths.

4.7.1 Loss function for volumetric cuts

Since the click position only correctly matches with the middle slice of the volume and the additional slices were used as supplementary information for the network, the correction loss needed to be adjusted.

For volumetric cuts, the loss function was composed of two components. The first component was the correction loss (defined in Equation 4.1) which was only computed for the middle slice, since that is the volume that needs to be corrected. The other component of the loss was the dice loss (defined in Equation 2.4), which was calculated for the whole volume. The total loss can be therefore defined by the following equation:

$$L_{total} = L_{corr} + \alpha L_{dice} \quad (4.2)$$

where α is a given constant. During experiments, α was set to 0.75 to underline the correction loss.

4.7.2 Results

In this part, we experimented with two different cut depths. In this case, we did not want to go below 8, as in this case, we would need to change the number of downsampling operations in the network architecture, which we view as not beneficial. On the other hand, we also did not want to use a higher depth because the tumour itself is relatively small (on average only occupies 10 slices). The resulting test DSC for the individual cut depths can be seen in Table 4.6.

	Testing
multi-encoder U-Net, volumetric correction loss, 8 cut depth	0.8010
multi-encoder U-Net, volumetric correction loss, 16 cut depth	0.8152
previous best model (multi-encoder U-Net)	0.8687

Table 4.6: Testing DSC for models trained on volumetric cuts of different depths and the best multi-encoder model.

From the results, the model trained with cuts of higher depth (16) performed slightly better than the model trained on cuts with depth 8. However, the added additional dimension to the training data did not improve the networks performance, and the best model trained with the volumetric cuts still underperformed in comparison to the model trained on 2D cuts.

4.8 Model fine-tuning

Since the performance of the correction model was still not satisfactory, we decided to fine-tune the best correction model that we had trained so far on real segmentations. We hypothesise that the method, which we used to imitate segmentation errors for training was perhaps too trivial and that the model simply learnt how to revert the used morphological operations, instead of truly correcting the imperfections. In this regard, we decided to additionally fine-tune the model on the segmentations generated by the segmentation model, trained in Section 4.4.

For this, we applied the following pipeline, which can be seen in Figure (4.19).



Figure 4.19: Diagram describing the fine-tuning pipeline.

The fine-tuning pipeline consists of 4 main steps. At first, we generate a segmenta-

tion by inputting the given image into the segmentation model. Then we generate clicks that mark the areas where the segmentation is insufficient. The whole process of simulating user clicks is described in Section "Simulating user interaction". After the clicks are generated, we cut the given volume. Here we use the same process as in the generation of training data (Section 4.5.2). Finally, we use the generated cuts to further train the correction model. In the end, we acquired 2375 training cuts and 594 validation cuts.

For the fine-tuning of the model, we used the same optimiser settings and the same overall training process as in the previous experiments.

4.8.1 Results

With this experiment, we wanted to further improve the best multi-encoder model by additionally training it on cuts generated from inaccurate segmentations. The results of this experiment can be seen in Table 4.7, where we compared the fine-tuned model with the best multi-encoder U-Net based model.

	Testing
previous best model (multi-encoder U-Net)	0.8687
fine-tuned multi-encoder U-Net	0.8938

Table 4.7: Testing DSC for the fine-tuned model and the "original" model.

As can be seen from the table and the example segmentation in Figure (4.20), fine-tuning has indeed improved the performance of the model, where the DSC for the corrected segmentation improved from 0.7884 to 0.9265.



Figure 4.20: An example of corrected cuts produced by the fine-tuned multi-encoder U-Net (bottom row) compared to the original segmentation (top row). In this particular case, the corrected cuts achieved DSC of 0.9265

4.9 Reconstruction of the segmentation

Final part of our method is the reconstruction of the segmentation, from the corrected cuts. For this we used a simple algorithm that iterated over the provided clicks and replaced the area of the segmentation denoted by the click with the corrected cut. The pseudocode for this algorithm is provided bellow.

Algorithm 2: Reconstruction of the segmentation

Data: `pred`, `click_coords`, `cuts`, `cut_size`

Result: Reconstructed volume

for `click_idx` **to** `len(click_coords)` **do**

`coords` $\leftarrow$ `click_coords[click_idx]`;

`cut` $\leftarrow$ `cuts[click_idx]`;

`z, y, x` $\leftarrow$ `coords[0]`, `coords[1]`, `coords[2]`;

`pred[0, z, y - cut_size : y + cut_size, x - cut_size : x + cut_size]` $\leftarrow$ `cut`;

end

4.9.1 Results

In this part, we combined the previously explored methods and tested the full proposed pipeline.

As first, we used the segmentation model to generate an initial segmentation, for which we simulated the users interaction and generated clicks on regions we wanted to refine. Subsequently, these regions were extracted from the initial segmentation and the acquired cuts were sent to the correction model, which corrected them. Finally, we replaced the old regions in the initial segmentation with the corrected cuts, for which we used the method described in this section. In this case, we only generated one set of clicks for each segmentation. The results from the testing are in Table 4.8.

	Testing
initial segmentation model	0.8030
after correction	0.8630

Table 4.8: Testing DSC for the whole pipeline.

Additionally, Table 4.9 shows the DSC for the initial segmentation, reconstructed segmentation and the reconstructed segmentation after 2nd iteration (where we generated a new set of clicks) for a selected volume.

	DSC
initial segmentation	0.7342
corrected segmentation	0.8606
corrected segmentation, 2nd iteration	0.8908

Table 4.9: DSC of the initial segmentation and reconstructed segmentation after 1st and 2nd iteration for selected volume.

Figure (4.21) shows the first four slices of the segmented tumour. The whole segmentation masks are in the Appendix ??.

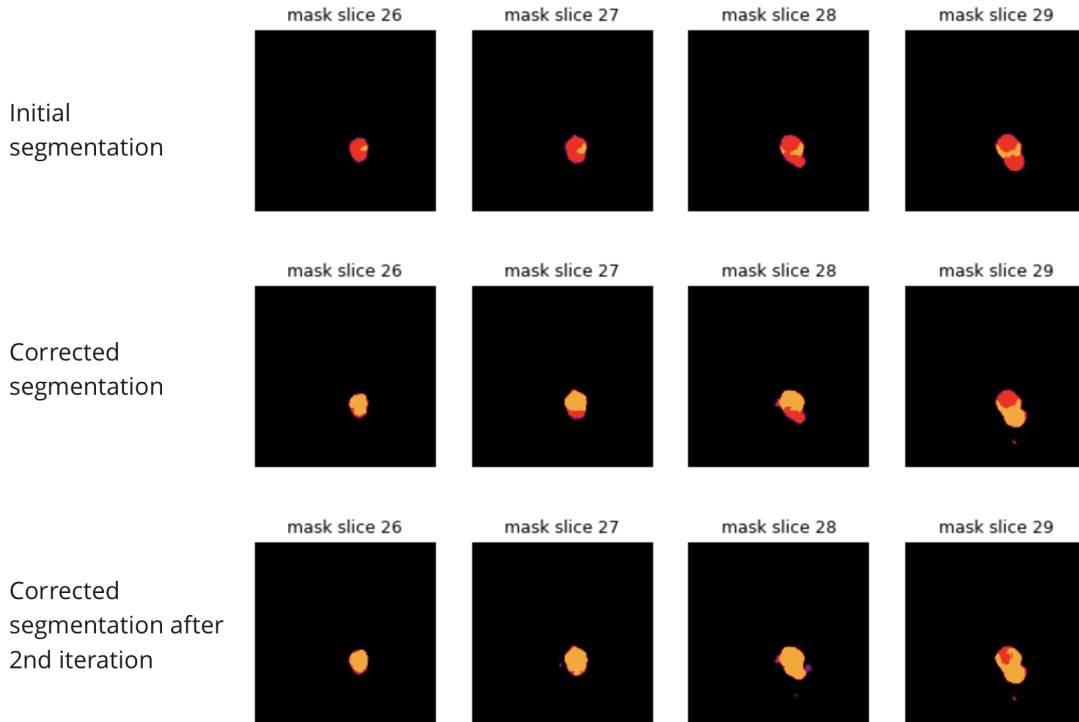


Figure 4.21: First four slices of a sample segmentation. The red colour denotes the ground truth label, the purple the generated segmentation and the orange the overlap between them.

Overall, the proposed reconstruction method is very naive and primarily serves as a demonstration of the process. In the future, it would be more appropriate to replace this method with a more complex process. One of the issues with this method is that it does not take into account the previous segmentations when replacing the cuts. This means that in some cases the correct segmentation might get overwritten by an incorrect one. A similar issue that might occur is that multiple cuts can sometimes involve the same region. Since the cuts are processed sequentially, later cuts might therefore overwrite regions that were corrected by the previous ones.

In this way it could be beneficial to expand the proposed reconstruction method with a strategy to handle the overlapping of certain regions.

Chapter 5

Conclusion

The objective of our work was to design a method that segmented Vestibular Schwannomas from T1ce and T2 MRI sequences, by utilising weakly-annotated data, in our case represented by clicks. The used approach was based on two networks; an initial segmentation network and a correction network. The proposed pipeline therefore consisted of the segmentation network, which generated the initial segmentation and the correction network, that based on the defined clicks extracted small regions from the initial segmentation and refined them. These refined regions replaced the original ones, resulting in an improved segmentation.

At first, we focused on the initial segmentation network. As the network architecture, we utilised the standard 3D U-Net. In this part, we based our experiments around finding the appropriate loss function that would allow us to use the least amount of data for training of the network, while still producing segmentations that are acceptable. Based on our experiments, we settled for a model trained on 64 images and **FocalTversky loss**, which achieved **0.8030** testing DSC. The fact that the model was over-fitted did not pose a problem for us as we could better test the abilities of our correction network.

Since the correction network uses small cutouts of the initial segmentation, we first developed a process to generate training data for this network, where we simulated faulty segmentations by using different morphological operations. At first, we based the architecture of the correction network on the Attention U-Net and experimented with different sizes of the cutouts as well as different loss functions. Eventually, the best-performing model was trained on 48-sized cuts and our custom modification of the dice loss function. This model achieved testing DSC of **0.8417**.

In the next stage, we hypothesise that the previous model was not fully utilising the additional information provided by the MRI sequences. To solve this issue, we employed a multi-encoder U-Net architecture, which consisted of two separated encoders, one for the faulty cut and one for stacked cutouts of the T1ce and T2 sequences. The separation of encoders slightly improved the performance of the model, which achieved testing DSC of **0.8687**.

In addition to the additional encoder, we also tried to train the network on volumetric cuts, where we experimented with different depths of the training volumes. The model however, did not benefit from the use of the additional dimension.

In order to further improve the performance of the multi-encoder model we decided to fine-tune it on "real" faulty segmentations, as opposed to the ones created by morphological operations. In this case, we utilised cuts extracted from a coarse segmentation generated by the initial segmentation network. These cuts were then used to fine-tune the model. The fine-tuning improved the models performance significantly and achieved testing DSC of **0.8938**.

In the end, we used a simple reconstruction method to demonstrate the complete pipeline, where corrected segmentations achieved testing DSC of **0.8630**, where in comparison the initial segmentations achieved DSC of 0.7538.

5.1 Future work

Our work could potentially be further improved by exploring different merging strategies in the multi-encoder correction model as the fusion of feature maps between the two modalities can have a major effect on the performance of the model.

Another possible improvement could be made in the segmentation reconstruction algorithm, which in its current form is too simplistic. A possible refinement of the method would be an addition of a strategy to handle the overlapping of certain regions, as we view this as a major step back.

Additionally, an interesting extension of the proposed method would be to use the correction network to also improve the initial segmentation model.

References

- [1] Andre Esteva et al. “Deep learning-enabled medical computer vision”. In: *NPJ digital medicine* 4.1 (2021), p. 5.
- [2] S. Kevin Zhou et al. “A Review of Deep Learning in Medical Imaging: Imaging Traits, Technology Trends, Case Studies With Progress Highlights, and Future Promises”. In: *Proceedings of the IEEE* 109.5 (2021), pp. 820–838. DOI: 10.1109/JPROC.2021.3054390.
- [3] Shapey Jonathan et al. *Segmentation of Vestibular Schwannoma from Magnetic Resonance Imaging: An Open Annotated Dataset and Baseline Algorithm*. [Data set]. 2021. DOI: doi.org/10.7937/TCIA.9YTJ-5Q73.
- [4] Roland Goldbrunner et al. “EANO guideline on the diagnosis and treatment of vestibular schwannoma”. In: *Neuro-Oncology* 22.1 (Oct. 2019), pp. 31–45. ISSN: 1522-8517. DOI: 10.1093/neuonc/noz153. eprint: <https://academic.oup.com/neuro-oncology/article-pdf/22/1/31/31789998/noz153.pdf>. URL: <https://doi.org/10.1093/neuonc/noz153>.
- [5] Sven-Eric Stangerup et al. “The natural history of vestibular schwannoma”. In: *Otology & Neurotology* 27.4 (2006), pp. 547–552.
- [6] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. 2015. arXiv: 1505.04597 [cs.CV].

- [7] Nahian Siddique et al. “U-Net and Its Variants for Medical Image Segmentation: A Review of Theory and Applications”. In: *IEEE Access* 9 (2021), pp. 82031–82057. DOI: 10.1109/ACCESS.2021.3086020.
- [8] Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi. “V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation”. In: *2016 Fourth International Conference on 3D Vision (3DV)*. 2016, pp. 565–571. DOI: 10.1109/3DV.2016.79.
- [9] Özgün Çiçek et al. *3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation*. 2016. arXiv: 1606.06650 [cs.CV].
- [10] Zongwei Zhou et al. “Unet++: A nested u-net architecture for medical image segmentation”. In: *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support: 4th International Workshop, DLMIA 2018, and 8th International Workshop, ML-CDS 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, September 20, 2018, Proceedings 4*. Springer. 2018, pp. 3–11.
- [11] Christian Szegedy et al. *Rethinking the Inception Architecture for Computer Vision*. 2015. arXiv: 1512.00567 [cs.CV].
- [12] Siddhartha Chandra et al. “Context aware 3D CNNs for brain tumor segmentation”. In: *Brainlesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries: 4th International Workshop, BrainLes 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, September 16, 2018, Revised Selected Papers, Part II 4*. Springer. 2019, pp. 299–310.
- [13] Rui Hua et al. “Multimodal brain tumor segmentation using cascaded V-Nets”. In: *Brainlesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries: 4th International Workshop, BrainLes 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, September 16, 2018, Revised Selected Papers, Part II 4*. Springer. 2019, pp. 49–60.

-
- [14] Ozan Oktay et al. *Attention U-Net: Learning Where to Look for the Pancreas*. 2018. arXiv: 1804.03999 [cs.CV].
 - [15] Mobarakol Islam et al. “Brain tumor segmentation and survival prediction using 3D attention UNet”. In: *Brainlesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries: 5th International Workshop, BrainLes 2019, Held in Conjunction with MICCAI 2019, Shenzhen, China, October 17, 2019, Revised Selected Papers, Part I* 5. Springer. 2020, pp. 262–272.
 - [16] Dhiraj Maji, Prarthana Sigedra, and Munendra Singh. “Attention Res-UNet with Guided Decoder for semantic segmentation of brain tumors”. In: *Biomedical Signal Processing and Control* 71 (2022), p. 103077.
 - [17] Hesheng Wang et al. “Automatic segmentation of vestibular schwannomas from T1-weighted MRI with a deep neural network”. In: *Radiation Oncology* 18.1 (2023), pp. 1–9.
 - [18] Jonathan Shapey et al. “An artificial intelligence framework for automatic segmentation and volumetry of vestibular schwannomas from contrast-enhanced T1-weighted and high-resolution T2-weighted MRI”. In: *Journal of neurosurgery* 134.1 (2019), pp. 171–179.
 - [19] Holger Roth et al. “Weakly Supervised Segmentation from Extreme Points”. In: *Large-Scale Annotation of Biomedical Data and Expert Label Synthesis and Hardware Aware Learning for Medical Imaging and Computer Assisted Intervention*. Ed. by Luping Zhou et al. Cham: Springer International Publishing, 2019, pp. 42–50. ISBN: 978-3-030-33642-4.
 - [20] Zongwei Zhou et al. “UNet++: Redesigning Skip Connections to Exploit Multiscale Features in Image Segmentation”. In: *IEEE Transactions on Medical Imaging* 39.6 (2020), pp. 1856–1867. DOI: 10.1109/TMI.2019.2959609.

- [21] Jiwoong J Jeong et al. “Systematic review of generative adversarial networks (gans) for medical image classification and segmentation”. In: *Journal of Digital Imaging* (2022), pp. 1–16.
- [22] Salome Kazeminia et al. “GANs for medical image analysis”. In: *Artificial Intelligence in Medicine* 109 (2020), p. 101938. ISSN: 0933-3657. DOI: <https://doi.org/10.1016/j.artmed.2020.101938>. URL: <https://www.sciencedirect.com/science/article/pii/S0933365719311510>.
- [23] Andreas Maier et al. “A gentle introduction to deep learning in medical image processing”. In: *Zeitschrift für Medizinische Physik* 29.2 (2019), pp. 86–101.
- [24] Thomas Neff et al. “Generative adversarial network based synthesis for supervised medical image segmentation”. In: *Proc. OAGM and ARW joint Workshop*. Vol. 3. 2017, p. 4.
- [25] Yuki Enokiya et al. “Automatic liver segmentation using U-Net with Wasserstein GANs”. In: *Journal of Image and Graphics* 6.2 (2018), pp. 152–159.
- [26] Saeid Asgari Taghanaki et al. “Deep semantic segmentation of natural and medical images: a review”. In: *Artificial Intelligence Review* 54.1 (2021), pp. 137–178.
- [27] Yassine Ouali, Céline Hudelot, and Myriam Tami. “An overview of deep semi-supervised learning”. In: *arXiv preprint arXiv:2006.05278* (2020).
- [28] Xiangli Yang et al. “A survey on deep semi-supervised learning”. In: *IEEE Transactions on Knowledge and Data Engineering* (2022).
- [29] Pawel Mlynarski et al. “Deep learning with mixed supervision for brain tumor segmentation”. In: *Journal of Medical Imaging* 6.3 (2019), pp. 034002–034002.
- [30] Longlong Jing, Yucheng Chen, and Yingli Tian. “Coarse-to-Fine Semantic Segmentation From Image-Level Labels”. In: *IEEE Transactions on Image*

- Processing* 29 (2020), pp. 225–236. ISSN: 1941-0042. DOI: 10.1109/TIP.2019.2926748.
- [31] Hoel Kervadec et al. “Bounding boxes for weakly supervised segmentation: Global constraints get close to full supervision”. In: *Proceedings of the Third Conference on Medical Imaging with Deep Learning*. Ed. by Tal Arbel et al. Vol. 121. Proceedings of Machine Learning Research. PMLR, 2020, pp. 365–381. URL: <https://proceedings.mlr.press/v121/kervadec20a.html>.
- [32] Kevis-Kokitsi Maninis et al. “Deep extreme cut: From extreme points to object segmentation”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018, pp. 616–625.
- [33] Holger R Roth et al. “Going to extremes: weakly supervised medical image segmentation”. In: *Machine Learning and Knowledge Extraction* 3.2 (2021), pp. 507–524.
- [34] Tianyi Zhao and Zhaozheng Yin. “Weakly Supervised Cell Segmentation by Point Annotation”. In: *IEEE Transactions on Medical Imaging* 40.10 (2021), pp. 2736–2747. DOI: 10.1109/TMI.2020.3046292.
- [35] Tomas Sakinis et al. “Interactive segmentation of medical images through fully convolutional neural networks”. In: *CoRR* abs/1903.08205 (2019). arXiv: 1903.08205. URL: <http://arxiv.org/abs/1903.08205>.
- [36] Qing En and Yuhong Guo. *Annotation by Clicks: A Point-Supervised Contrastive Variance Method for Medical Semantic Segmentation*. 2022. arXiv: 2212.08774 [cs.CV].
- [37] Ning Xu et al. “Deep Interactive Object Selection”. In: *CoRR* abs/1603.04042 (2016). arXiv: 1603.04042. URL: <http://arxiv.org/abs/1603.04042>.
- [38] Mostafa Jahanifar et al. “Robust interactive semantic segmentation of pathology images with minimal user input”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2021, pp. 674–683.

- [39] Zhanghexuan Ji et al. “Scribble-based hierarchical weakly supervised learning for brain tumor segmentation”. In: *Medical Image Computing and Computer Assisted Intervention–MICCAI 2019: 22nd International Conference, Shenzhen, China, October 13–17, 2019, Proceedings, Part III* 22. Springer. 2019, pp. 175–183.
- [40] Han Liu et al. “Unsupervised Domain Adaptation for Vestibular Schwannoma and Cochlea Segmentation via Semi-supervised Learning and Label Fusion”. In: *Brainlesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries*. Springer International Publishing, 2022, pp. 529–539. DOI: 10.1007/978-3-031-09002-8_46.
- [41] Kibrom Berihu Girum et al. “Fast interactive medical image segmentation with weakly supervised deep learning method”. In: *International Journal of Computer Assisted Radiology and Surgery* 15 (2020), pp. 1437–1444.
- [42] Xiaokang Li et al. “HAL-IA: A Hybrid Active Learning framework using Interactive Annotation for medical image segmentation”. In: *Medical Image Analysis* (2023), p. 102862.
- [43] Dhruv Sharma et al. “Active learning technique for multimodal brain tumor segmentation using limited labeled images”. In: *Domain Adaptation and Representation Transfer and Medical Image Learning with Less Labels and Imperfect Data: First MICCAI Workshop, DART 2019, and First International Workshop, MIL3ID 2019, Shenzhen, Held in Conjunction with MICCAI 2019, Shenzhen, China, October 13 and 17, 2019, Proceedings 1*. Springer. 2019, pp. 148–156.
- [44] Mélanie Gaillochet, Christian Desrosiers, and Hervé Lombaert. “TAAL: Test-time augmentation for active learning in medical image segmentation”. In: *MICCAI Workshop on Data Augmentation, Labelling, and Imperfections*. Springer. 2022, pp. 43–53.

-
- [45] Alessandro Bria, Claudio Marrocco, and Francesco Tortorella. “Addressing class imbalance in deep learning for small lesion detection on medical images”. In: *Computers in Biology and Medicine* 120 (2020), p. 103735. ISSN: 0010-4825. DOI: <https://doi.org/10.1016/j.compbimed.2020.103735>.
- [46] Shruti Jadon. “A survey of loss functions for semantic segmentation”. In: *2020 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology (CIBCB)*. 2020, pp. 1–7. DOI: 10.1109/CIBCB48159.2020.9277638.
- [47] Jun Ma et al. “Loss odyssey in medical image segmentation”. In: *Medical Image Analysis* 71 (2021), p. 102035. ISSN: 1361-8415. DOI: <https://doi.org/10.1016/j.media.2021.102035>.
- [48] Michael Yeung et al. “Unified Focal loss: Generalising Dice and cross entropy-based losses to handle class imbalanced medical image segmentation”. In: *Computerized Medical Imaging and Graphics* 95 (2022), p. 102026. ISSN: 0895-6111. DOI: <https://doi.org/10.1016/j.compmedimag.2021.102026>.
- [49] Jeroen Bertels et al. “Optimization with soft dice can lead to a volumetric bias”. In: *Brainlesion: Glioma, Multiple Sclerosis, Stroke and Traumatic Brain Injuries: 5th International Workshop, BrainLes 2019, Held in Conjunction with MICCAI 2019, Shenzhen, China, October 17, 2019, Revised Selected Papers, Part I* 5. Springer. 2020, pp. 89–97.
- [50] Saeid Asgari Taghanaki et al. “Combo loss: Handling input and output imbalance in multi-organ segmentation”. In: *Computerized Medical Imaging and Graphics* 75 (2019), pp. 24–33.
- [51] Wentao Zhu et al. “AnatomyNet: deep learning for fast and fully automated whole-volume segmentation of head and neck anatomy”. In: *Medical physics* 46.2 (2019), pp. 576–589.

- [52] Carole H Sudre et al. “Generalised dice overlap as a deep learning loss function for highly unbalanced segmentations”. In: *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support: Third International Workshop, DLMIA 2017, and 7th International Workshop, ML-CDS 2017, Held in Conjunction with MICCAI 2017, Québec City, QC, Canada, September 14, Proceedings 3*. Springer. 2017, pp. 240–248.
- [53] Shruti Jadon. “A survey of loss functions for semantic segmentation”. In: *2020 IEEE Conference on Computational Intelligence in Bioinformatics and Computational Biology (CIBCB)*. 2020, pp. 1–7. DOI: 10.1109/CIBCB48159.2020.9277638.
- [54] Nabila Abraham and Naimul Mefraz Khan. “A Novel Focal Tversky Loss Function With Improved Attention U-Net for Lesion Segmentation”. In: *2019 IEEE 16th International Symposium on Biomedical Imaging (ISBI 2019)*. 2019, pp. 683–687. DOI: 10.1109/ISBI.2019.8759329.
- [55] Hao Guan and Mingxia Liu. “Domain adaptation for medical image analysis: a survey”. In: *IEEE Transactions on Biomedical Engineering* 69.3 (2021), pp. 1173–1185.
- [56] Hyungseob Shin et al. *Self-Training Based Unsupervised Cross-Modality Domain Adaptation for Vestibular Schwannoma and Cochlea Segmentation*. 2021. DOI: 10.48550/ARXIV.2109.10674. URL: <https://arxiv.org/abs/2109.10674>.
- [57] Jae Won Choi. *Using Out-of-the-Box Frameworks for Contrastive Unpaired Image Translation for Vestibular Schwannoma and Cochlea Segmentation: An approach for the crossMoDA Challenge*. 2021. DOI: 10.48550/ARXIV.2110.01607. URL: <https://arxiv.org/abs/2110.01607>.
- [58] Jun-Yan Zhu et al. *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks*. 2017. DOI: 10.48550/ARXIV.1703.10593.

-
- [59] Fan Xu et al. “LSTM Multi-modal UNet for Brain Tumor Segmentation”. In: *2019 IEEE 4th International Conference on Image, Vision and Computing (ICIVC)*. 2019, pp. 236–240. DOI: 10.1109/ICIVC47709.2019.8981027.
- [60] Matej Halinkovič. “Interpretability and Explainability of Deep Learning Systems in the Domain of Microscopic Medical Imaging”. pp. 58-60. Master’s Thesis. Bratislava: FIIT STU, 2023.
- [61] Guotai Wang et al. “Interactive Medical Image Segmentation Using Deep Learning With Image-Specific Fine Tuning”. In: *IEEE Transactions on Medical Imaging* 37.7 (2018), pp. 1562–1573. DOI: 10.1109/TMI.2018.2791721.
- [62] Yigit B. Can et al. “Learning to Segment Medical Images with Scribble-Supervision Alone”. In: *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support*. Ed. by Danail Stoyanov et al. Cham: Springer International Publishing, 2018, pp. 236–244. ISBN: 978-3-030-00889-5.
- [63] Fei Du et al. “Efficient mask correction for click-based interactive image segmentation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2023, pp. 22773–22782.
- [64] Konstantin Sofiiuk, Ilya A Petrov, and Anton Konushin. “Reviving iterative training with mask guidance for interactive segmentation”. In: *2022 IEEE International Conference on Image Processing (ICIP)*. IEEE. 2022, pp. 3141–3145.
- [65] Xi Chen et al. “Focalclick: Towards practical interactive image segmentation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 1300–1309.
- [66] François Chollet. “Xception: Deep learning with depthwise separable convolutions”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017, pp. 1251–1258.

- [67] Konstantin Sofiiuk, Olga Barinova, and Anton Konushin. “Adaptis: Adaptive instance selection network”. In: *Proceedings of the IEEE/CVF international conference on computer vision*. 2019, pp. 7355–7363.
- [68] David Martin et al. “A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics”. In: *Proceedings Eighth IEEE International Conference on Computer Vision. ICCV 2001*. Vol. 2. IEEE. 2001, pp. 416–423.
- [69] Federico Perazzi et al. “A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016.
- [70] Shoukun Sun et al. “Cfr-icl: Cascade-forward refinement with iterative click loss for interactive image segmentation”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 38. 5. 2024, pp. 5017–5024.
- [71] Qin Liu et al. “SimpleClick: Interactive Image Segmentation with Simple Vision Transformers”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023, pp. 22290–22300.
- [72] Ke Sun et al. “Deep High-Resolution Representation Learning for Human Pose Estimation”. In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019, pp. 5686–5696. DOI: 10.1109/CVPR.2019.00584.
- [73] Xiaoming Liu et al. “Weakly supervised segmentation of COVID19 infection with scribble annotation on CT images”. In: *Pattern recognition* 122 (2022), p. 108341.
- [74] Mostafa S Ibrahim et al. “Semi-supervised semantic image segmentation with self-correcting networks”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 12715–12725.

- [75] Kun Li, George Vosselman, and Michael Ying Yang. “Interactive Image Segmentation with Cross-Modality Vision Transformers”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023, pp. 762–772.
- [76] Jiacheng Lin et al. “AdaptiveClick: Click-Aware Transformer With Adaptive Focal Loss for Interactive Image Segmentation”. In: *IEEE Transactions on Neural Networks and Learning Systems* (2024).

